One Table: A Studio Footage Pipeline Run by One Person

Oscar Martinez
MOG Systems, Inc.

Abstract

A film studio moves footage through a chain of specialists. Asset managers and production trackers gather much of that work into one catalogue, but each covers its own stages in its own database, and the offload, the catalogue, the tracker, the edit and the publishing tools still keep separate records. This paper describes a production system that runs post production, analysis and publishing for one person, around the steps it leaves to people on purpose, by keeping all of those records in a single replicated log and one read model built from it.

The system runs on the state primitive specified in the companion paper [1]: schema validation, hierarchical routing and access control are relations over a single five token subject key, `tenant.<tenant>.dag.<family>.<id>`. A record's family is a total function of its immutable schema identifier. The JetStream stream that stores a family is bound to that family's branch of the subject tree, and every identity's grant is a list of patterns over the same tree. Every check on a write runs in the writer's process with no call to another service; the write itself is one hop, to the broker, which repeats the grant and stream checks. Reads against the PostgreSQL read model are bound to the caller's tenant in a single pipelined round trip under forced row security policies. Agents take work by subscription, and people collaborate in shared CRDT drafts that settle into the same log.

The deployment carries a creator's studio footage pipeline (11,967 clips, 2,441 GB in vault storage) under its own tenant root. Because nothing in the state kernel is specific to video, the registry also names two more tenant kinds: a multi tenant software product, and a vertically integrated retail store. The store's record families are specified as the design's scope, and the deployment runs the creator. The studio pipeline runs on a local NAS, two on premises GPU machines, Macs running DaVinci Resolve against a shared PostgreSQL project library, and a cloud account costing $1,169.90 per thirty days before language models.

1. Introduction

> "Leveraging jet stream to build a DAG that powers all the data shapes we use and because of the policies you can apply it becomes a self selective and self contained data registry where only the right policy grants access. Couple that with an event driven architecture & collaboration becomes easy. You can plug in humans and use CRDTs to keep UIs eventful or just subscriptions to keep agents running and iterating."

A studio offloads a camera card to two places and verifies both copies before the card is cleared [2], [3]. An assistant editor makes light copies of each clip, called proxies, so the editor can cut without the heavy originals, and the finished cut is conformed to the originals and rendered from them [4]. Quality control (QC) checks the render before it ships. An asset manager catalogues every file and where it is stored [5], [6].

Each of those steps exists because footage is heavy and easy to lose. Each step also keeps its own list. The lists disagree, and people spend time making them agree.

This system keeps one list, and one person runs it: I direct and approve the work, and agents and model services do the rest. Every step writes to it, and every step reads from it. Its steps follow the practice of film and television post production. The table below maps each studio step to its counterpart here, and says where the counterpart is weaker than the practice.

Studio step	Here
Offload to two verified copies before the card is cleared	Each clip is copied from the card to the NAS, checked by size, then to the NAS archive tier and to a cloud vault, an S3 bucket. The archive copy is checked by its size and by the clip's identity hash, which covers the first and last 16 MiB of a large file, and the vault

Studio step	Here
	copy by its size. The card may be cleared when both are confirmed. No copy of a large clip is checked against a hash of every byte (section 11)
Proxies for the edit	ProRes 422 Proxy files at half resolution, with the same file stem, frame count and timecode as the original, made on the Macs. Where none exists, the small analysis copy is linked in its place
Render from originals	The master is rendered in Resolve from the timeline's media, which is linked to the camera originals on the NAS, and the platform copies are encoded from that master
Asset manager	The log, with an edge from each render to the clips it was cut from
QC	A measured pass on each render, run as a job of its own, on the hook's placement, the audio's loudness and peaks, black and flash frames, and the render's size
Conform and edit decision lists	No edit decision list leaves Resolve. The edit and the render share one Resolve project library [7], and a relink by file name to the NAS takes the place of a conform

The log itself, with the proof that its write check is bounded and the measurements of that check, is the subject of the companion paper, *One Name for Schema, Hierarchy and Access Control on a Replicated Log* [1]. This paper describes the studio built on it: the list, the boundary around it, the steps that use it, what it costs, and the boundaries of what it claims.

What exists already. None of the studio steps here is new, and most of them are sold as products. CatDV indexes a library held on site, in the cloud or on tape in one place and runs workers that automate ingest, transcoding and file movement [5]. Iconik's storage gateway watches a NAS and indexes the files in place, makes proxies and keyframes on local hardware, and tags every asset with machine learning [6]. EditShare FLOW makes proxies while it ingests and adds transcription and shot detection [8], and Axle AI runs its tagging and search on premises against existing storage [9]. The sidecar of section 5 and the analysis of section 6 do what these already do. Flow Production Tracking, formerly ShotGrid, writes an event log entry for every action on a site, and a daemon hands each event to plugins, without guaranteed delivery [10]. AYON stores events with the event that caused them, and its services claim work through a call that creates the result only once for each source event [11]. Avid MediaCentral runs jobs from rules with triggers [12]. The jobs of section 8 and the workers that take work by subscription follow the same pattern on a broker. Frame.io's Camera to Cloud uploads proxies before the originals arrive [13], as the phone in section 5 does, and ASC MHL records the hashes and the history of every copy of a card as a chain of custody [14]; my sidecar writes no such record. Handing out work by subscription, and creating the next job from a rule, are therefore not new, and I do not claim them. What I claim is narrower. First, one replicated log is the system of record for every stage, from the card to the weekly read and people's decisions, and the read model is a fold of that log, rebuilt by replaying it; the systems above keep their catalogue or their event log in a database of their own, and their documentation describes no rebuild from a log. Second, the log is named as the companion paper describes, with the tenant in every record's address and each record's stream fixed by its schema, so the studio's records, their access rules and their streams share one name.

2. The state kernel and the log

Records and families. Everything the business tracks is a record: a camera clip, a pose or vision tag, a post, a human decision, a job, or a run. Every record is a JSON envelope carrying a `node_id` or `edge_id`, an immutable versioned `schema_id` (`sch_<Name>_v<N>`), and a `payload`. Records of one kind form a family. The reference implementation carries 57 schemas across 28 families: 24 record families that a creator tenant carries, the family of work queue messages, and three families of the platform root.

The replicated log. Records are stored in NATS JetStream on a three node cluster with three replicas per stream at the hub, each stream ordered by its own Raft group. Every record is published to a five token subject:

`tenant.<tenant>.dag.<family>.<id>`

A program called the projector folds the log into a PostgreSQL read model, which the services read.

One naming across four structures. Schemas, permissions and data are not stored in one table. Four separate structures are indexed by the same five token subject [1]:

1. **The schema registry** is an in memory map from `schema_id` to compiled JSON Schema (draft 2020 12), held in every writer and in the projector. Each binary embeds a baseline snapshot as a cold start floor. The projector and the writers built on the ledger library also follow the platform root's schema stream (`tenant.<platform>.dag.schema.<id>`), so new schemas and families enter them with no restart. The bridge that writes records from the GPU machines and the Macs validates against the schemas embedded in its build.
2. **The stream topology** binds at most one stream per family under each tenant root (`tenant.<tenant>.dag.<family>.>`). The family is `Kind(schema_id)`, a total function of the schema identifier, resolved in 3.8 nanoseconds. A writer cannot choose a record's family.
3. **The grant table** is a set of `(identity, verb, pattern)` tuples rendered into the broker configuration and evaluated against the five token subject trie.
4. **The row policy** in PostgreSQL enforces `tenant_id = (SELECT mog_tenant())` on every table of records in the read model, where `tenant_id` is taken from token 2 of the subject the record arrived on, never from the writer's self reported body.

A write's checks take 225 nanoseconds on the name plus local validation, then one publish. Before publishing, the writer derives the family and subject from `schema_id`, matches the identity's grant trie, resolves the target stream, and validates the payload against the compiled schema. The three name lookups take 225 nanoseconds in process; validating a 726 byte asset record takes 15.2 microseconds [1]. The write is then one publish to the broker, which repeats the grant and stream checks. An unknown schema identifier maps to the `unknown` family, which binds no stream, so both the local check and the broker refuse it with no call to a separate registry. The projector folds a record only at its own address: a subject of exactly five tokens, whose family token is `Kind(schema_id)` and the family of the stream it arrived on, and whose last token is the record's own id. It refuses any other.

Facts versus records. General worker processes hold no publish grant on record families. When a worker finishes a task, it publishes a fact on its tenant's ingress branch (`...dag.ingress.>`), and the authority, the service that writes records on behalf of people and workers, turns verified facts into records. Every call to the authority passes through the web service in front of it, which sets the tenant from the caller's session or key on every route. On the run, exception, resolution and pack routes it also stamps the identity and the actor from the credential, and refuses a body that carries them. On the copy routes it stamps the actor from the credential and overwrites any actor in the body. Specialized media nodes hold narrow record grants: the GPU workers may publish directly to the asset and derivation edge families, and the Resolve Macs may publish to those two families plus the validation family for QC findings [1].

Deterministic ids and the read model fold. A derived id is 120 bits (20 base64url characters): SHA 256 over the record's defining fields, truncated. A clip's id is the full SHA 256 of its sampled content hash (section 11), its file name and its size. Writing the same logical record twice produces the same id. The projector consumes all streams in parallel and folds them into seven PostgreSQL tables: nodes, edges, tombstones, and four derived tables. Every row stores the `(stream, epoch, sequence)` position of the message that wrote it, acting as a last writer wins register ordered by log position, and keeps the earliest `created_at` that any record of the id carries since its last deletion.

Empirical rebuild parity. Because the read model is a deterministic fold of the log, a lost database is rebuilt from the retained log. Replaying the first tenant's production log of 1,454,979 messages into an empty scratch database on 20 October 2026, at fold version 4, completed in 536 seconds (117 s read, 408 s apply, 11 s compare) with the following parity against the live read model [1]:

Table	Rebuilt	Live	Written after read	Outside retained log	Only live	Only rebuilt	Contents differ
Nodes	293,707	293,947	53	196	0	0	45
Edges	334,449	334,665	22	194	0	0	0
Tombstones	148	148	0	0	0	0	0

Zero rows exist only in the rebuild, and every edge and tombstone the rebuild holds matches its live row. The 45 node differences are all in `created_at`, the live value earlier than the rebuilt one: either the row remembers a

write the retained log does not hold, or a retraction and a later rewrite of the same id arrived out of log order; the run did not separate the two per row [1, section 6.7]. Those 45 rows, and the 196 nodes and 194 edges outside the retained log, stay in the live model and are not restored by a rebuild. They are the boundary of a rebuild from a retained log that the companion paper states [1, section 11]. The columns do not sum to the live count: 293,707 plus 53 plus 196 is 293,956, nine more than 293,947, because nine of the 53 rows past the read heads were updates to rows that both models hold, and the tombstone table shows no deletion past the heads.

3. Tenants and reads

A tenant is one business on the system: its records, its rules, its people and its files. Each tenant is a root in the subject tree. A second root, the platform root, holds the schema records, the tenant records and the user records. The bootstrap, a program with its own identity, writes those records and creates each tenant's streams.

A membership joins one identity to one tenant in one role: manager, talent, team or agent. A manager may hold memberships in several tenants.

Credentials. A person signs in through Amazon Cognito's hosted page. When the token is issued, a function stamps into it the tenants the person holds, and the console's edge forwards them. A machine's or an agent's key carries its tenant and memberships in the key table.

The rules for every read.

1. The tenant comes from the credential, never from the request. A caller with several memberships selects one of them. It cannot name a tenant it does not hold.
2. A credential that names no tenant gets nothing.
3. A list returns the selected tenant's records only.
4. A read of another tenant's record answers exactly as a read of a missing record does, so ids cannot be probed.

At the broker. Each worker identity's grants name its own tenant's root and no other tenant's. A worker may create only its own named consumer, filtered to its own tenant's subjects, and each identity receives replies only on its own inbox. The authority and the projector bind streams and never create them. The bootstrap creates a tenant's streams when the tenant, or a family its kind carries, is added. Each tenant after the first has its own streams, named for the tenant and the family, so two roots never share storage. One identity reaches everything, and it exists for emergencies. A read only diagnostic identity may fetch any stored message.

In the read model. Every row carries its tenant. Postgres row security policies [15] on every table of records compare the row's tenant with the tenant set for the current transaction. The services read under a role that cannot bypass those policies. Each request sets its tenant from the credential when its transaction begins. A query that runs with no tenant set fails with an error. It does not return every row.

In the file store. The signer that hands out a download link looks the record up through the tenant bound read. It signs only an id the caller's tenant holds. Another tenant's id answers as a missing one does. Each tenant after the first has its own prefix in the vault. The vault blocks public access and encrypts every object at rest with keys the storage service manages.

The change stream. The console keeps a live connection that announces every change. Each subscriber receives only its own tenant's changes. A change with no tenant is dropped.

The proof. Two mock tenants are defined beside the creator: a home cooking channel and a trail running channel. Each has a copy agent, a pack importer and a mock Mac as members; its mock manager and mock talent are test identities. Neither has a social account. A suite of 35 probes, 67 checks in all, runs each probe in both directions where it has one, among them the broker, the read model, the file store, the change stream, the shared drafts, the job system, the agents' memory and inference. Of the 67, 56 are attempts by one tenant on the other, and each must end in its named refusal; 10 are controls that must succeed, and 1 checks the projector's order of an edge and a retraction. A probe that cannot run is recorded as skipped, never as passed.

The suite runs in the ledger's build on a local stack: an embedded broker with the bootstrap's grants, an embedded Postgres, the real projector, and reads under the reader role. That stack has no web service, vault or shared drafts, so the probes on those planes skip there. On 19 October 2026, in the ledger's build, the result was:

Result	Checks
Passed	16
Failed	0
Skipped	51

The passing checks cover the broker's grants, the projector's refusal of a record under the wrong tenant and the row policies; one of them is the order check.

The teardown deletes a mock's streams, its rows in the read model, its vault prefix and its memberships, and closes its tenant record.

4. People and agents

Agents and machines take work by subscription. A worker subscribes to the task subjects it is granted. When a record it depends on is written, it receives the work, does it, and reports the result as a fact. Each worker writes a presence beat every 15 seconds. A worker silent for 45 seconds is marked stale.

People take part through shared drafts. A CRDT is a data structure that merges edits made at the same time by different people, without a server deciding their order. The drafts here are Yjs documents [16]. Each object people work on together has its own room: the tags on a clip, the copy for a post, and the post itself, where agents' proposals wait. A room belongs to one tenant. Joining it checks the caller's membership and role before the connection opens.

Everyone in a room edits the same draft and sees the others' edits as they type. When someone settles a draft, the web service takes the draft as it holds it, never the caller's copy, and the authority runs the checks any write of that kind runs and writes a record to the log. A copy record's id is computed from the tenant, the post and the copy's content, so a settle repeated after a dropped connection writes nothing new. The settled state is a part of the room that only the server writes.

The draft is never written to the log and never read back when the log is replayed. Losing every draft loses unsaved edits and nothing else. The log stays the only source.

A room is loaded when it is first used, by a connection joining it or a route writing to it, and is evicted from memory after five minutes with no use, unless it holds a pending proposal, a live claim or a settle in flight. On eviction its stored updates are compacted to one update per document. A room that nobody has loaded or written for thirty days is dropped from the store, and the first use of a tenant loads its stored post rooms, so a pending proposal survives a restart.

Agents join the copy and post rooms with their own tenant bound keys; a clip's tag room is for people only, and agents read a post room without editing it. An agent may draft copy and settle it as a candidate, and picking a candidate is a separate command. An agent may propose an edit point, a stage move or a copy pick. Only a person settles a proposal.

The console is the people's view. It is a static web page that reads the same records the agents read. It lists each post at its stage, the decisions waiting for a person, the failures, the footage and the machines. There are thirteen kinds of decision. Some are proposed by an agent; the board raises the rest from the records, such as a stalled job or a post that underperformed. Only a person settles one.

5. Ingest

A camera card is copied into an inbox folder on the NAS by a copy job on the NAS, which checks each file's size against the card. A program called the sidecar walks the inbox from a GPU machine on the same network, five minutes after each pass ends.

For each clip the sidecar computes an identity hash, makes an analysis copy, uploads the small copies to the vault, uploads the original last, and writes the clip's record. It then checks the vault copy's size, moves the clip to the originals folder, copies it to the archive tier, and checks that copy against the hash. It writes a small file beside the clip that says whether the vault copy and the archive copy are confirmed. The card may be cleared when both are. If the sidecar cannot write a camera clip's record, it does not move the clip. The sidecar cannot

file a clip without the vault: it asks the vault for the original before anything else, so while the line to the cloud is down, clips wait in the inbox and are filed on the first pass after it returns.

The analysis copy is made on the GPU's video encoder, at 1,024 pixels on the long edge, with the audio in a separate 16 kHz mono file. The Macs make the ProRes Proxy copies for editing.

An upload to the vault that the sidecar has not processed starts a second pass in the cloud. A container builds a smaller video copy and still frames at two per second, and marks the dead time in the clip. It trims the dead time only when the upload asks. The pass runs on zero to eight containers, depending on the queue.

A phone is a second camera. The phone app makes a proxy, uploads it to the vault, and the clip's record is created from it. The original travels separately: to the vault, or, on the local network, to a small receiver on the NAS. It takes an upload in 8 MiB parts, checks a SHA 256 of the file when the upload completes, and writes the clip into the phone's inbox on the hot share with a small record beside it. The ingest tests whatever lands there before anything is recorded. The server computes the record's id from the capture itself: the tenant, the file's original name and its capture time to the second. The original carries the same three facts, so it lands on the record the proxy created, with no relinking and no lookup between the two. Before filing an original, the sidecar checks that the name, the size and both computed ids agree. On any disagreement it files nothing. An original that reaches only the vault has that one copy until a job brings it to the NAS.

On 18 October 2026 the vault held 2,441 GB. On 19 October the creator tenant's records listed 11,967 clips.

6. Analysis

Each clip is read twice by machines before a person looks at it.

The first read measures the body. A pose model, YOLO11 nano with the 17 COCO keypoints [17], [18], tracks the joints frame by frame. The system counts a repetition when a body signal, a joint angle or the distance between two joints, crosses a threshold and comes back; where the signal is an angle, it records the range of motion in degrees. The thresholds were set on one card of 56 clips, on which the clips with an implausible count of repetitions fell from 41 to 5. This paper does not report the counts against a hand count.

The second read describes the clip. An 8B vision language model, Qwen3-VL 8B Instruct [19], runs on an RTX 4090 in the room. It looks at up to six frames and answers a fixed list of questions, among them the exercise, the body part, the shot, the camera angle, the motion and, where I have named places, the room. It also writes a caption of two or three sentences. Each answer is stored as a tag with its source and its confidence. This paper does not score the tags against a person's reading.

Phone clips usually carry GPS; the camera's clips carry none. Where a clip has coordinates, the system matches them to places I have named.

Search runs on the tags, the captions among them, with Postgres full text search. The system does not use vector embeddings for search.

7. The edit

DaVinci Resolve [23] composes every video. The system does not replace it. It prepares everything Resolve needs and drives Resolve through its scripting interface.

Each post has one folder and one job file. The job file names the post, its format, the Resolve project, the timeline and the name of the render. Each Resolve step can run as a job of its own: build, hook, voice, captions, plate, in points, anatomy, render, QC and transcode, with a prepare job that runs the steps before the build and a finish job that runs the steps after the render. The transcode makes the platform copies with ffmpeg, outside Resolve. A worker on a Mac runs one step per job.

Resolve's scripting interface drives a running copy of Resolve Studio. So the render machine is a Mac that keeps Resolve open in a logged in session and takes one job at a time from a queue. The design calls for two. Every copy of Resolve on the network opens the same project library, a PostgreSQL project server, so a timeline built on the render Mac is the same timeline I open on my own Mac.

The edit uses the proxies. The master is rendered from the timeline's media, linked to the originals. The measured QC pass of section 1 runs on each render before anyone approves it.

Every render mutes the camera's sound, which carries the gym's music. The only audio on a post is the voice and a licensed music bed. The voice is my recording or a clone of my voice, and each post records which.

8. Jobs, runs and exceptions

Work is a job. A job is a record that names its inputs, its checks and its outputs. A rule creates the next job when its inputs exist, and a sweep applies the rule every thirty seconds by default. A person may order a step. A runner takes the job: a program, a model, a Mac's browser, a Resolve Mac, or a person. Model jobs and browser jobs then run with no person to start them.

Each attempt is a run, and each run is a record written by the runner, never by the program being run. A job carries a timeout and a retry count by its class, so a runner that goes silent times its job out. A run that fails or times out, or a check that fails, raises an exception: a stored record of the failure. An exception stays open on the console until it is resolved or superseded by a newer one. A retry that passes resolves it, and only a person may skip, settle or dismiss one. An agent may be handed an exception to fix twice by default. After that it goes to a person as a decision.

A model job reads only what its job type lists: named fields of the tenant's rules, named texts and named files. The tenant's rules are records, imported from the documents on every merge that touches them. Each run of a model job records what it loaded.

Each model job runs at a level from 0 to 3. A setting can name the model for each level, and overrides a default table in the code; each run records which one it used. The cloud levels run on small cloud language models. The vision model runs on the local GPU, with a cloud GPU kept as failover.

9. The weekly loop

The creator tenant is the worked example in this paper. Its loop is one week long. A week starts with a plan and ends with a reading.

The plan places the week's posts in five fixed positions. The number of posts follows a ramp of four, five, six and seven a week. Each step is taken only after four weeks with no missed post, and a missed week drops back one rung.

Each post moves through eleven stages: ideated, scripted, copied, retrieved, prepared, composed, rendered, checked, approved, published, measured. The stage gate checks the audio from prepared on, the QC pass at checked, and the disclosure, the rating and the sources at approval; each job records its own checks. A panel of model readers, each a language model playing one kind of viewer, ranks each post's copy. A clear winner goes forward. A close call comes to me.

A post reaches the platforms through a browser job on the Mac that holds the account logins. A video becomes a dated draft in a scheduling service, which I publish from its planner. An X post is posted on x.com by the job itself. A YouTube video is uploaded through YouTube Studio by a job of its own. A later job reads each draft's state back and records it.

After a post is published, the system reads its numbers from each platform's own analytics and writes one record per publication for each source and each day it is read. At the end of the week each combination of topic, format and delivery gets one of three verdicts: drop it, keep it, or make more of it. The verdict shapes the next plan.

A rung counts posts; a rerun or a loop of one master is a publication of its own. The time from a card to a published post is not measured.

10. Three kinds of tenant

Nothing in the state kernel of section 2, or in the boundary of section 3, is specific to footage. The registry stores families of records, and a family is whatever a business needs to track. A tenant's record names its kind, and each schema record names the kinds that carry its family. The registry names three kinds.

Kind	Families it carries
A creator	The shared set below, and nine of its own: rules, experiments, ideas, decompositions, patterns, references, search terms, annotations and places
A software product with its customers	The shared set
A store	The shared set

The shared set is decisions, jobs, runs, exceptions, resolutions, the rules pack, settings and edges, with clips, tags, posts, copy, validations, publications and engagement for the tenant's own marketing.

A family enters as one record. A family the store needs, such as a purchase order, is added by one schema record that names the family and the kind store. The bootstrap then creates the family's stream under each store tenant, and under no creator. No program is rebuilt or redeployed. A test does exactly this: after a purchase order schema is appended, a store tenant gets the streams of its sixteen families and no creator's, a purchase order is accepted under the store's root, and the same record is refused under a creator's.

The store is the furthest from footage, so it is the test. These are the main families it needs; the full specification has twenty. None of them is in the registry.

Area	Family	One record is
Products and bill of materials	`product`	One style in one colour: its sizes, its specification, its status
	`bom`	The parts one unit needs, with quantities: fabric, elastic, thread, labels, packaging
Suppliers and purchase orders	`supplier`	One maker or mill: its terms and lead times
	`purchase_order`	One order to a supplier: lines by style and size, dates, state
	`shipment`	Goods in transit: what was sent, what was counted on arrival
Inventory and size curve	`stock_move`	One movement of units of one item: received, sold, returned, written off. Stock on hand is the sum
	`size_curve`	The planned split of a style across sizes, beside the split that sold
Orders and fulfilment	`order`	One customer order, as Shopify reports it
	`fulfilment`	Picking, packing, the carrier's tracking, delivery, a return
Customer service	`ticket`	One customer conversation and how it ended
Marketing and affiliates	`campaign`	One paid or organic push: its window, its spend, its results
	`affiliate`	One partner: its code and its terms
	`payout`	One payment owed or made to a partner
Quality and cost from the factory	`production_run`	One factory run of a style: units made, defects found by kind, measurements against the specification, cost per unit against the bill of materials

Records refer to each other through edges, as clips and renders do. A purchase order points to its supplier. A stock move points to the shipment or the order that caused it. A production run points to its style and its bill of materials. The question "which fabric lot produced the units in this return" is a walk along edges, the same walk that traces a render back to its clips.

An outside system enters the way a worker does. A program publishes its events as facts on the tenant's ingress subjects, and the authority turns them into records. For the store, that program is a Shopify connector carrying orders, fulfilments and refunds. The connector is part of the store's specification, and the deployment runs the creator tenant only. The store's jobs are to follow the creator's rules: a shipment counted short is to raise an exception, as any failed check does.

Onboarding a tenant creates one stream under the tenant's root for each family the kind carries, then writes a tenant record on the platform root that names its kind, and a user record for each member, which carries the memberships. The tenant's machine identities are a change to the grants, and a person's tenants are stamped into the token from a list in the infrastructure definitions, so adding a person is an infrastructure change too. The broker, the projector, the read model, the console, the job system and the boundary in section 3 are the ones the creator tenant uses. What is new for a store is its own programs: the connector, the jobs that act on its records, and the screens that show them, designed to run in a cloud account of their own.

11. What stays human, and the boundaries

Four things are left to people on purpose. The boundaries of what this paper claims follow them.

The first is Resolve. I judged the effects and depth maps built outside Resolve worse than what a person makes in it. So Resolve stays, and the system works around it.

The second is the cut. An agent that reviews a cut proposes changes. Every change I made stays. A timeline that holds my layered takes is never rebuilt.

The third is the settling of what agents propose. Agents propose stages, edit points and copy picks, and a person settles each proposal. A person also settles every close call between copy candidates, and every skip, settle or dismissal of an exception. Agents pick copy when the panel's result is clear, and programs move a post when its checks pass, so a post can reach approval with no person; an X post can.

The fourth is the publish of a video: the system makes the dated draft, and I send it.

The boundaries of the primitive. The companion paper states six boundaries of the state primitive [1, section 11]. Two of them bound what this paper reports:

1. **Coarse subject grants.** Broker grants evaluate a verb against the tenant and family tokens of the five token name in bounded time and never traverse derivation edges. Rules that follow lineage are made by the authority before it writes; the one recursive walk over edges, the console's tree of posts, stops at 50 levels.
2. **Retained log containment.** Streams the bootstrap creates are capped: 8 MiB for a mock tenant and 256 MiB for any other. A record stream never drops a record: at its cap it refuses new writes and the publish fails at the writer, deleting or purging a message is denied, and an alarm fires when a stream reaches 80 percent of its cap. Replay parity is containment: every row the rebuild produces is in the live model. Rows whose first writes the retained log does not hold, all written before these settings, remain in Postgres, and the rebuild does not reproduce them. In the run of section 2 they are the 196 nodes and 194 edges outside the retained log and the 45 nodes whose creation time differs.

The boundaries of the deployment. Seven belong to the deployment:

1. **Single account root isolation.** All tenants run inside one NATS account, isolated by their subject roots (`tenant.<tenant>.dag.>`), dedicated per family streams, and identity grants rather than separate NATS accounts. This is a deployment choice, not a property of the primitive.
2. **Head and tail media hashing.** To avoid a full read of multi gigabyte 4K camera originals at ingest, the sidecar hashes a clip's first 16 MiB, its last 16 MiB and its size with 128 bit xxh3; a file of 32 MiB or less is hashed whole. The clip's identity is that hash paired with the original file's name and its size in bytes, and its node id is minted from the three; the capture time is not part of it. So two clips of the same length whose middle frames differ share an identity only if they also share a file name. That case is the boundary. Camera file names carry the camera's own date, time and counter, which is why the pairing holds in practice. The same hash is the only content check on the archive copy, so damage between the first and last 16 MiB of a large clip is not detected.
3. **Resolve timeline execution.** Resolve's scripting interface drives a running copy of Resolve Studio. Resolve also runs on Linux [23], and the scripting documentation installed with it describes a mode without the interface, but this deployment runs it on logged in macOS machines against a shared PostgreSQL project library, and complex grading stays with the human editor.
4. **Disconnected edge storage.** The two GPU workers run as sites: each runs a local NATS leaf server on loopback with an 8 GB file store [1, section 10], so the worker's asset and derivation edge records are acknowledged at the site during a WAN outage. Ingest does not continue, because the sidecar must reach the vault before it files a clip (section 5). No Mac runs a leaf. In the companion paper's tests, sourcing back to the hub took 0.14 to 2.2 seconds once the link was up, and in the two site runs every record was at the hub 2.2 to 4.4 seconds after the line returned. The hub's source from each site is filtered to that site's families. No line to either site has been cut in the deployment, so these figures are the test's.

5. **Quorum loss.** While the cluster has no majority, the broker refuses its clients: writes fail rather than diverge, as the companion's section 6.1 states. A rolling update holds each replacement node until it has joined the cluster and its streams are current, and replaces a node that hangs, before it takes the next.
6. **The NAS.** The NAS holds the proxies, the camera originals and their archive copies. The hot share is on a mirrored pair of NVMe drives, and the archive is on the spinning disks, so a clip's NAS original and its archive copy sit on separate disks in one box. The Resolve project library runs on the second GPU machine and is dumped to the NAS hourly and daily, the daily dumps kept thirty days. The camera originals have a second copy in the vault; the proxies and the project dumps are kept on the NAS.
7. **Flush interval.** The brokers keep JetStream's default flush interval. A write is acknowledged once a majority of its stream's replicas have written it, and each replica forces its writes to disk at the interval, two minutes by default. Acknowledged writes can be lost if every replica loses power within that window.

12. Cost

The heavy work runs on machines in the room. The vision model and the image work run on an RTX 4090; the pose model runs on it and on a second machine with an RTX 3060. The cloud GPU groups sit at zero machines and start one only when the queue backs up. A g5.2xlarge, the cloud card kept as failover, lists at $1.212 an hour, or $884.76 for a month of 730 hours [20]. A used 4090 averaged $2,527 in the thirty days to 17 September 2026 [21]; October listings start near $3,000 [22].

When the vision model moved from the cloud to the 4090, the cloud GPU line went from 331 hours ($401.56) in September to 12.5 hours ($15.11) in the first seventeen days of October.

From 5 to 17 October, thirteen days, the cloud account cost $1,169.90 per thirty days before language models, each line scaled by 30/13. About two thirds of that is networking, its firewall rules and monitoring; the machines and containers are $195.16.

Line	Per 30 days
Traffic between availability zones	$176.38
Monitoring	$174.67
Instances	$110.99
Load balancers	$104.55
Private endpoints	$102.22
NAT gateway	$97.38
Containers	$84.17
Firewall rules	$73.12
Everything else	$246.42
Total before language models	**$1,169.90**

About $60 of the figure is services outside the pipeline. The broker machines run at a larger size from 13 October; at that size for the whole period, the figure is about $1,203.

Language models are billed apart. Nearly all of that bill is the interactive sessions that build and run the system, which cost $343.36 from 1 to 17 October, and the bill does not separate building the system from running it. The pipeline's own copy drafts measured in it cost $1.03 over the same days.

At four posts a week the account's cost is $68 a post. At seven it is $39. Footage is processed per clip, and one clip serves several posts. So one more post made from footage already stored adds, in the cloud, the copy drafts, the panel's reads and the voice: a few cents for the measured drafts. These figures are the cloud bill only, before language models. They leave out the hardware, software licences and subscriptions, electricity and internet. My time is not measured.

The smallest cloud setup that runs this system costs $215 a month at list price, before storage, logs and language models. It is three t4g.medium broker machines with their volumes, one small database machine, one small control server for the private network, two containers, one load balancer, one NAT gateway, four public addresses and a few secrets. It leaves out traffic between zones and through the NAT, which a cluster across three zones incurs.

A tenant adds records, streams and files to that setup, not broker or database machines. But JetStream reserves each stream's cap in full on every replica: a store's sixteen families reserve 4 GiB on every broker and a creator's twenty four 6 GiB, against a file store of 18 GB per broker, so past two or three tenants the brokers' disks must grow.

13. How to build one

Build it in the order a clip travels. Check each stage with a read before starting the next.

1. The ledger: a three node NATS JetStream cluster, a NAT gateway for the containers' traffic out, the schemas, the grants, the authority, the projector into Postgres, and the row policies.
2. Ingest: an S3 vault with an archive rule that covers the originals' folder, the sidecar on any machine with an NVIDIA GPU, and a NAS with a fast tier and an archive tier.
3. Analysis: a pose model and a small vision model on the local GPU.
4. The edit: Resolve Studio on three Macs, because Resolve runs here only on Macs that keep it open, and a shared project server.
5. Jobs, the console and the weekly loop.
6. A second tenant, and the probes of section 3 between the two.

I would build the ledger, the two copy offload and the probes first; the browser machines, the phone app and the cloud GPU groups can wait.

14. Open specification

MOG Systems, Inc. publishes this specification alongside the companion formal paper [1] and the runnable benchmark suite (`primbench`) [24]. The core architecture collapses schema validation, hierarchical stream routing, and identity access control under a single canonical subject key over a replicated log.

15. Conclusion

The system keeps one ledger: one replicated log and one read model in place of the departments' separate lists. The schema registry, the subject tree and the grants share one five token key, so a kind of data, its access rule and its event stream are named together. That is one naming, not one store. The tenant comes from the credential, and the broker and the database refuse what the credential does not cover. Machines take work by subscription. People edit shared drafts that settle into the same log. The registry names a creator, a software product and a store as kinds of tenant, and a new kind of record enters as one schema record. The work a studio spreads across several departments runs here on a NAS, two GPU machines, a few Macs and a cloud account that costs about $1,170 every thirty days before language models. The boundaries are stated in section 11.

Appendix: where each claim is checked

Every statement of fact above was checked against the implementation on 20 October 2026, a read of the log, the bill, or a public source. The implementation is not published; the benchmark [24] reproduces the write check and the bound read only. The table names each component, what it does, and where a reader can check its claim.

Component	What it does	Where the claim is checked
The write check	Derives the family and subject from the schema id, matches the grant, finds the stream, validates the record	[1], sections 3 and 8; the benchmark [24]
The schema registry	Holds every schema locally and follows new ones from the platform root's schema stream	[1], section 2.3
The replicated log	Stores every record in a stream per family, three replicas, one Raft group per stream	[1], sections 2.2 and 6.1
The grants	Name, per identity, the subjects it may publish to and subscribe to	[1], sections 2.4 and 4
The read model and its row policy	Folds the log into Postgres and binds every read to the credential's tenant	[1], sections 2.5, 3.4 and 8; the benchmark [24]

Component	What it does	Where the claim is checked
The fold and the rebuild	Orders each row by log position, writes tombstones, rebuilds from an empty database	[1], sections 6.4 to 6.7
The tenant boundary	Refuses every read and write across tenants on every plane	The probe suite, section 3 of this paper
Shared drafts	Merge people's edits and settle them into the log	[1], section 6.6; section 4 of this paper
The site leaf	Accepts writes during a disconnection and sources them to the hub	[1], section 10
Ingest, analysis, the edit, jobs and the weekly loop	Move a clip from the card to a published post	The implementation on 20 October 2026, read for this paper
Cost	The account's bill and list prices	The cloud provider's cost report for 5 to 17 October 2026, scaled to 30 days; list prices [20]; hardware prices [21], [22]

References

1. O. Martinez. *One Name for Schema, Hierarchy and Access Control on a Replicated Log.* MOG Systems, Inc. The companion paper. https://mog.systems/papers/one-name/SPEC.html
2. Newsshooter. *Stow: free copy and verification software.* https://www.newsshooter.com/?p=315376
3. Pomfort. *Pomfort introduces the Silverstack Offload Manager.* https://pomfort.com/article/pomfort-introduces-the-silverstack-offload-manager/
4. S. Goldin. *8 Pitfalls to Avoid in an Offline-Online Video Workflow.* Frame.io, 10 December 2018. https://blog.frame.io/2018/12/10/offline-online-workflow-pitfalls/
5. Quantum. *CatDV asset management.* https://www.quantum.com/en/products/asset-management/
6. Iconik. *Hybrid cloud.* https://www.iconik.io/hybrid-cloud
7. Blackmagic Design. *DaVinci Resolve collaboration.* https://www.blackmagicdesign.com/products/davinciresolve/collaboration
8. EditShare. *FLOW.* https://www.editshare.com/products/flow
9. Axle AI. *Axle AI.* https://www.axle.ai/
10. Autodesk. *shotgunEvents: the Flow Production Tracking event daemon.* https://github.com/shotgunsoftware/shotgunEvents/wiki
11. Ynput. *AYON event system.* https://docs.ayon.dev/docs/dev_event_system
12. Avid. *MediaCentral.* https://www.avid.com/products/mediacentral
13. Frame.io. *Camera to Cloud.* https://frame.io/c2c
14. ASC Motion Imaging Technology Council. *ASC MHL.* https://github.com/ascmitc/mhl
15. PostgreSQL. *Row Security Policies*, PostgreSQL 18 documentation, section 5.9. https://www.postgresql.org/docs/18/ddl-rowsecurity.html
16. Yjs. *Yjs: a CRDT framework.* https://docs.yjs.dev
17. Ultralytics. *Ultralytics YOLO11.* https://docs.ultralytics.com/models/yolo11/
18. Ultralytics. *Pose estimation.* https://docs.ultralytics.com/tasks/pose/
19. Qwen Team. *Qwen3-VL.* https://github.com/QwenLM/Qwen3-VL
20. Amazon Web Services. *Amazon EC2 On-Demand Pricing*, US East (N. Virginia), read through the AWS Price List API on 18 October 2026. https://aws.amazon.com/ec2/pricing/on-demand/
21. RunAIHome. *Used RTX 4090 vs New RTX 5090 for Local AI in 2026.* 17 September 2026. https://runaihome.com/blog/used-rtx-4090-vs-new-rtx-5090-2026/
22. GPU Poet. *NVIDIA GeForce RTX 4090 Price in October 2026.* https://gpupoet.com/gpu/learn/price/october-2026/nvidia-geforce-rtx-4090
23. Blackmagic Design. *DaVinci Resolve.* https://www.blackmagicdesign.com/products/davinciresolve
24. MOG Systems, Inc. *primbench: the benchmark.* https://github.com/MOG-Systems-Inc/primbench